



Communication-Efficient Secure Neural Network via Key-Reduced Distributed Comparison Function

Peng Yang¹, Zoe Lin Jiang¹(✉), Shiqi Gao¹, Hongxiao Wang², Jun Zhou³,
Yangyiye Jin³, Siu-Ming Yiu², and Junbin Fang⁴

¹ Harbin Institute of Technology, ShenZhen, China
{stuyangpeng, 200111514}@stu.hit.edu.cn, zoejiang@hit.edu.cn

² The University of Hong Kong, Hong Kong, China
{hxwang, smyiui}@cs.hku.hk

³ Shanghai Pudong Development Bank, Shanghai, China
{zhoujun, jinyyy}@spdb.com.cn

⁴ Jinan University, Guangzhou, China
tjunbinfang@jnu.edu.cn

Abstract. In privacy-preserving neural network, the high communication costs of securely computing non-linear functions is the primary performance bottleneck. For commonly used non-linear functions, such as ReLU, existing methods adopt an offline-online computation paradigm and utilizes distributed comparison function (DCF) to reduce communication costs. Specifically, these methods prepare DCF keys in the offline phase and perform secure non-linear function computation using these keys in the online phase. However, the practicality of these methods is limited due to the substantial size of DCF keys and the heavy reliance on a trusted third party during the offline phase.

In this work, we introduce FssNN, a communication-efficient secure two-party neural network framework, which features a key-reduced DCF scheme without a trusted third party to enable practical secure training and inference. Firstly, by analyzing the correlations between DCF keys to eliminate redundant parameters, we propose a key-reduced DCF scheme with a compact additive construction, decreasing the size of DCF keys by about 17.9% and offline communication costs by approximately 28.0%. Secondly, leveraging an MPC-friendly pseudorandom number generator, we propose a secure two-party distributed key generation protocol for our key-reduced DCF, eliminating the need for a trusted third party. Finally, we utilize the key-reduced DCF and additive secret sharing to compute non-linear and linear functions, and design secure computation protocols with constant online communication rounds for neural network operations, reducing online communication costs by 28.9%–43.4%.

We provide formal security proofs and evaluate the performance of FssNN on various models and datasets. Experimental results show that compared to the state-of-the-art framework AriaNN, our framework reduces the total communication costs of secure training and inference by approximately 25.4% and 26.4% respectively.

Keywords: Privacy-preserving neural network · Secure multi-party computation · Distributed comparison function · Additive secret sharing

1 Introduction

Machine learning using neural networks is widely applied in many practical scenarios such as healthcare prediction, financial services and policy making. To improve the models' accuracy and generalization capability, many organizations wish to collect data from multiple parties to jointly train neural network models, but privacy concerns and regulations prevent them from openly sharing their data [20]. Privacy-preserving neural network based on secure multi-party computation (MPC), such as homomorphic encryption [12], garbled circuit [28] and secret sharing [14], provides a promising solution by allowing different parties to train various models on their joint data without revealing any sensitive information. Compared with privacy-preserving neural network based on homomorphic encryption and garbled circuit, privacy-preserving neural network based on secret sharing has significant advantages in computation efficiency and communication efficiency respectively, making it widely used for neural network training and inference in the real-life scenarios [11].

However, compared to plaintext neural network training, privacy-preserving neural network based on secret sharing still incurs substantial communication costs. The main source of inefficiency in prior work [20, 21, 24] is that the bulk of secure computation of non-linear functions in neural network, such as ReLU and MaxPool. Within secure computation protocols, these non-linear functions need to be represented as Boolean and arithmetic circuits, which have a high circuit depth, and the communication complexity of securely computing these nonlinear functions is proportional to the circuit depth [8]. Therefore, the high communication costs of secure non-linear functions computation protocols is the primary performance bottleneck in privacy-preserving neural network. To decrease the communication costs of secure non-linear function computation protocols, existing work [2, 15, 16, 22] adopts an offline-online computation paradigm [7] and utilizes distributed comparison function (DCF) [3, 4] to design efficient protocols. Specifically, these works precompute DCF keys in the offline phase and perform secure non-linear function computation using these DCF keys in the online phase. However, these work requires generating a large number of DCF keys in the offline phase and relies on a trusted third party to generation DCF keys, incurring massive communication costs in the training and inference process.

To address these issues, we introduce a communication-efficient secure two-party neural network framework, called FssNN, to enable practical secure training and inference. First, by analyzing the correlations between DCF keys to eliminate redundant parameters, we propose a key-reduced DCF scheme with a compact additive construction, which decreases the size of DCF keys by about 17.9% and the offline communication costs by approximately 28.0%. Secondly, by leveraging an MPC-friendly pseudorandom number generator (PRG) [9], we

propose a secure two-party distributed DCF key generation protocol for our key-reduced DCF, thereby eliminating the reliance on the trusted third party and supporting a larger input domain ($\mathbb{Z}_{2^{32}}$ and above) than the state-of-the-art DCF key generation scheme [2]. Finally, we utilize our key-reduced DCF and additive secret sharing to compute non-linear and linear functions, respectively, and design secure computation protocols with constant online communication rounds for neural network operators, reducing the online communication costs by 28.9%–43.4%.

Theoretical analysis indicates that, compared with existing work ABY2.0 [21] and AriaNN [22], our FssNN achieves fewer online communication rounds and lower communication costs. Experimental results show that, compared to the state-of-the-art framework AriaNN [22], the communication costs for secure training and inference in FssNN is reduced by approximately 25.4% and 26.4%, respectively, with the accuracy difference between secure and plaintext training and inference being only about 0.05%.

1.1 Related Work

Privacy-preserving neural network built on MPC has emerged as a flourishing research area in the past few years. Existing works use secure computation protocols based on secret sharing to compute linear functions and protocols based on secret sharing (SS), garbled circuit (GC), or function secret sharing (FSS) to compute non-linear functions. These works also adopt the offline-online computation mode [7] to obtain an efficient online phase by moving a majority of computation and communication costs to the offline phase.

In SS-based and GC-based solutions, SecureML [20] is the first privacy-preserving neural network framework and implementation with secure two-party computation (2PC) based on SS and GC. It enables the secure training by combining Boolean secret sharing, arithmetic secret sharing and Yao’s secret sharing [8], but the conversion between three types of secret shares incurs huge communication costs. ABY2.0 [21] reduces online communication rounds and communication costs by designing an efficient secret shares conversion protocol, and improves the efficiency of secure two-party neural network. However, these 2PC frameworks have a number of communication rounds linear to the circuit depth, resulting in extremely high communication costs and latency. ABY3 [19] and Falcon [25] are proposed to tackle secure training by leveraging three-party computation (3PC), and Trident [6] and Tetrad [17] are the secure four-party computation (4PC) framework for privacy-preserving neural network training. Compared with 2PC frameworks, these 3PC and 4PC frameworks have fewer communication rounds (still linear with circuit depth) [11], but impose a stronger security assumption (honest-majority rather than the dishonest-majority), the practicality is greatly limited.

In FSS-based solutions, non-linear functions are evaluated by using FSS-based 2PC protocols, which are optimal in terms of online communication and rounds [2]. AriaNN [22] is a low-interaction privacy-preserving neural network framework based on FSS by reducing the key sizes of distributed comparison

function (DCF, a FSS scheme for comparisons) However, it still requires considerable online communication costs, and the secure protocols within AriaNN designed for ReLU incorporate a 1-bit error. BCG+21 [2], LLAMA [15] and Grotto [23] provide secure computation protocols based on DCF for computing various math functions (e.g., comparison, reciprocal square root and piecewise polynomial), yet they currently can not provide support for training neural networks. Orca [16] enables secure inference and training by accelerating the computation of FSS-based 2PC protocols with GPUs, but the online phase in Orca requires additional communication rounds compared with other FSS-based solutions, incurring the high communication latency. However, AriaNN [22], LLAMA [15], Grotto [23] and Orca [16] rely heavily on a unrealistic trusted party to generate DCF key in the offline phase. Although BCG+21 [2] designs a distributed DCF key generation scheme by extending the Doerner-Shelat protocol [10] to replace the trusted party, it is only suitable to a small input domain ($\mathbb{Z}_{2^{16}}$ or smaller), which greatly limits its practicality. The FSS-based secure neural network frameworks are summarized in Table 1.

Table 1. The FSS-based secure neural network frameworks.

Framework	Training	Inference	Online Gate Evaluation Rounds	Without Trusted Party	GPUs
BCG+21 [2]	×	×	1	✓	×
AriaNN [22]	✓	✓	1	×	✓
LLAMA [15]	✓	×	1	×	×
Grotto [23]	×	×	1	×	×
Orca [16]	✓	✓	$O(1)$	×	✓
FssNN (Ours)	✓	✓	1	✓	×

1.2 Our Contributions

We propose a communication-efficient secure two-party neural network framework called FssNN, which to enable practical secure training and inference. In details, our contributions can be summarized in the following three aspects:

- **Key-reduced DCF with a compact additive construction.** By analyzing the correlations between DCF keys to eliminate redundant parameters, we propose a key-reduced DCF scheme with a compact additive construction with fewer key sizes than the state-of-the-art DCF construction [2] from $n(\lambda + 3) + \lambda + 1$ bits to $(\lceil n - \log \lambda \rceil)(\lambda + 3) + 2\lambda$ bits where n is the input size and λ is the security parameter, thus reduce offline communication costs by 26.8%–28.3%.
- **Distributed DCF key generation based on MPC-friendly pseudorandom generators.** by leveraging an MPC-friendly pseudorandom number generator, we propose a secure two-party DCF distributed key generation protocol for our key-reduced DCF, thereby eliminating the reliance on a trusted third party and supporting a larger input domain ($\mathbb{Z}_{2^{32}}$ and above) than the state-of-the-art DCF key generation scheme [2].

- **Online efficient secure neural network operators.** By utilizing the proposed key-reduced DCF and additive secret sharing, we construct online-efficient secure computation protocols for neural network operators, such as matrix multiplication, Hadamard product, ReLU and Maxpool, and reduce the online communication costs by 43.4% and 28.9%, respectively.

2 Preliminaries

Notations. $\mathbb{Z}_{2^n}$ is a ring with arithmetic operations with each element identified by its n -bit binary representation. We parse $x \in \{0, 1\}^n$ as $x_{n-1} || \cdots || x_0$ where $x_{[i]} \in \mathbb{Z}_2$ denotes x_i and $x_{[i,j]} \in \mathbb{Z}_{2^{j-i}}$ denotes the ring element corresponding to the bit-string $x_{j-1} || \cdots || x_i$ for $0 \leq i < j \leq n$. Denote scalar, vector and matrix by lowercase letter x , lowercase bold letter $\mathbf{x}$ and uppercase bold letter $\mathbf{X}$ respectively. Denote security parameter by λ , and $\mathbf{1}\{b\}$ by the indicator function that outputs 1 when b is true and 0 otherwise.

2.1 Neural Network

Let $D = \{(\mathbf{x}_i, y_i) | i \in \{0, 1, \dots, m\}\}$ denotes training datasets where each data sample $\mathbf{x}_i$ contains d features with the corresponding output label y_i . Neural network is a computational process to learn a function g such that $g(\mathbf{x}_i) \approx y_i$ where g can be represented as a function of weight matrix $\mathbf{W}$ and input data $\mathbf{x}_i$. The neural network training procedure consists of two phases, namely forward propagation and backward propagation. The phase to calculate the predicted output $\hat{y}_i = g(\mathbf{W}, \mathbf{x}_i)$ is called *forward propagation*, which comprises of linear operations and a non-linear activation function. One of the most popular activation functions is the rectified linear unit (ReLU).

To learn the weight $\mathbf{W}$, a cost function $C(\mathbf{W})$ that quantifies the error between predicted value $\hat{y}_i$ and actual value y_i is defined, and $\mathbf{W}$ is calculated and updated by solving the optimization problem of $\text{argmin}_{\mathbf{W}} C(\mathbf{W})$. The solution for this optimization problem can be computed by using stochastic gradient descent (SGD), which is an effective approximation algorithm for approaching a local minimum of a function step by step. SGD algorithm works as follows: $\mathbf{W}$ is initialized as a vector of random values or all 0s. In each iteration, a sample $(\mathbf{x}_i, y_i)$ is selected randomly and the coefficient matrix $\mathbf{W}$ is updated by $\mathbf{W} := \mathbf{W} - \alpha \nabla C(\mathbf{W})$, where α is the learning rate and $\nabla C(\mathbf{W})$ is the partial derivatives of the cost with respect to the changes in weight. The phase to calculate the change $\alpha \nabla C(\mathbf{W})$ is called *backward propagation*, where error rates are fed back through a neural network to update weight $\mathbf{W}$.

In practice, instead of selecting one sample of data per iteration, a small batch of samples are selected randomly and $\mathbf{W}$ is updated by averaging the partial derivatives of all samples on the current $\mathbf{W}$. This is called a mini-batch SGD, and its advantage is to allow for the use of vectorization techniques.

2.2 Additive Secret Sharing

An additive secret sharing (SS) scheme splits a secret value into multiple shares that add up to the original secret, ensuring no individual share reveals the secret. In a two-party SS scheme, party P_0 with secret share $\langle x \rangle_0$ and party P_1 with secret share $\langle x \rangle_1$ share the secret value $x \in \mathbb{Z}_{2^n}$, such that $x = (\langle x \rangle_0 + \langle x \rangle_1) \bmod 2^n$. We say that P_0 and P_1 together hold the secret share pair $\langle x \rangle$, meaning that P_0 holds $\langle x \rangle_0$ and P_1 holds $\langle x \rangle_1$.

Sharing and Reconstruction. To realize the functionality $\mathcal{F}_{\text{Share}}$ which additively shares a secret value $x \in \mathbb{Z}_{2^n}$, secret owner samples random $r \in \mathbb{Z}_{2^n}$, and sends $\langle x \rangle_b = (x - r) \bmod 2^n$ to P_b and sends $\langle x \rangle_{1-b} = r$ to P_{1-b} . To implement the functionality $\mathcal{F}_{\text{Recon}}$ which opens an additively shared value $\langle x \rangle$, P_b sends $\langle x \rangle_b$ to P_{1-b} who computes $(\langle x \rangle_0 + \langle x \rangle_1) \bmod 2^n$ for $b = \{0, 1\}$. In the following text, we omit the modular operation for simplicity.

Addition and Multiplication. Functionalities $\mathcal{F}_{\text{Add}}$ and $\mathcal{F}_{\text{Mul}}$ add and multiply two shared values $\langle x \rangle$ and $\langle y \rangle$ respectively. It is easy to non-interactively add the shared values by having P_b compute $\langle z \rangle_b = \langle x \rangle_b + \langle y \rangle_b$. We overload the addition operation to denote the secure addition by $\langle x \rangle + \langle y \rangle$. To realize $\mathcal{F}_{\text{Mul}}$, taking the advantage of Beaver's precomputed multiplication triples technique [1], the specific protocol Π_{Mul} works as follows: assume that P_0 and P_1 hold multiplication triples $\langle u \rangle, \langle v \rangle, \langle uv \rangle$ where $u, v \in_R \mathbb{Z}_{2^n}$, P_b locally computes $\langle e \rangle_b = \langle x \rangle_b - \langle u \rangle_b$ and $\langle f \rangle_b = \langle y \rangle_b - \langle v \rangle_b$ and then the two parties reconstruct $\langle e \rangle, \langle f \rangle$ to get e, f . Finally, P_b lets $\langle z \rangle_b = b \cdot e \cdot f + f \cdot \langle u \rangle_b + e \cdot \langle v \rangle_b + \langle uv \rangle_b$.

Specifically, functionality $\mathcal{F}_{\text{Gen}^{\text{MT}}}$ generates multiplication triples $(\langle u \rangle, \langle v \rangle, \langle uv \rangle)$ consumed in Π_{Mul} . Typically, multiplication triples can be generated based on oblivious transfer (OT). In this paper, the protocol $\Pi_{\text{Gen}^{\text{MT}}}$ for $\mathcal{F}_{\text{Gen}^{\text{MT}}}$ is achieved by directly using OT-based generation scheme [20, 26].

In the case of $n > 1$ (e.g., $n = 32$) which supports arithmetic operations (e.g., addition and multiplication), arithmetic share pair is denoted by $\langle \cdot \rangle^A$. In the case of $n = 1$ which supports Boolean operations (e.g., XOR and AND), Boolean share pair is denoted by $\langle \cdot \rangle^B$. In this paper, we mostly use the arithmetic share pair and denote it by $\langle \cdot \rangle$ for short.

2.3 Function Secret Sharing

Intuitively, a two-party function secret sharing (FSS) scheme [2] splits a function $f \in \mathcal{F}$ into two shares f_0, f_1 , such that: (1) each f_b hides f ; (2) for each input x , $f_0(x) + f_1(x) = f(x)$. This section follows the definition of FSS from [2].

Definition 1. A two-party FSS scheme is a pair of algorithms $(\text{Gen}, \text{Eval})$:

- (1) $\text{Gen}(1^\lambda, \hat{f})$ is a probabilistic polynomial-time (PPT) key generation algorithm that given 1^λ and $\hat{f} \in \{0, 1\}^*$ (description of a function $f : \mathbb{G}^{\text{in}} \rightarrow \mathbb{G}^{\text{out}}$) outputs a pair of keys (k_0, k_1) . We assume that $\hat{f}$ explicitly contains descriptions of input and output groups $\mathbb{G}^{\text{in}}, \mathbb{G}^{\text{out}}$.

- (2) $\text{Eval}(b, k_b, x)$ is a polynomial-time evaluation algorithm that given $b \in \{0, 1\}$ (party index), k_b (key defining $f_b : \mathbb{G}^{\text{in}} \rightarrow \mathbb{G}^{\text{out}}$) and $x \in \mathbb{G}^{\text{in}}$ (input for f_b) outputs a group element $y_b \in \mathbb{G}^{\text{out}}$ (the value of $f_b(x)$).

Distributed Comparison Function and Its Variant. A special piecewise function, $f_{\alpha, \beta}^<(x)$, also referred to as a comparison function, outputs β if $x < \alpha$ and 0 otherwise. We refer to a FSS scheme for comparison functions as distributed comparison function (DCF). And the variant of DCF, called dual distributed comparison function (DDCF), is considered and denoted by $f_{\alpha, \beta_0, \beta_1}^<(x)$ that outputs β_0 for $0 \leq x < \alpha$ and β_1 for $x \geq \alpha$. Obviously, $f_{\alpha, \beta_0, \beta_1}^<(x) = \beta_1 + f_{\alpha, \beta_0 - \beta_1}^<(x)$ and thus DDCF can be constructed by DCF.

Secure Two-Party Computation via FSS. Recent work of Boyle et al. [2, 5] shows that FSS paradigm can be used to efficiently evaluate some function families in the two-party computation in the offline-online model, where Gen and Eval correspond to the offline phase and the online phase respectively. Refer [2] for more details.

2.4 Threat Model

We consider two-party computation secure against a *semi-honest* adversary, i.e., the corrupted party running the protocol honestly while trying to learn as much information as possible about others' input or function share. In this paper, we directly follow the definition of semi-honest security from [18].

In addition, modular sequential composition theorems [13, 18] are considered, and we prove protocols secure under the definition of semi-honest security from [18] and immediately derive their security under sequential composition. Our protocols invoke several sub-protocols and for ease of exposition we describe them using the hybrid model, which is the same as a real interaction except that the sub-protocol executions are replaced with calls to the corresponding trusted functionalities - protocol invoking $\mathcal{F}$ is said to be in the $\mathcal{F}$ -hybrid model.

3 The Proposed FssNN

In this section, we present a high-level overview of FssNN framework in Subsect. 3.1, and provide detailed construction of secure linear layer functions and secure non-linear layer functions in Subsect. 3.2 and Subsect. 3.3 respectively. Due to space limitations, the security proof of the proposed protocols is in the full version of this paper [27].

3.1 The FssNN Overview

In FssNN, we consider parties P_0 and P_1 who want to train various models on their joint data. We denote the training datasets D_0 and D_1 holding by parties P_0 and P_1 respectively. To distribute the secret shares of training datasets between

P_0 and P_1 , P_0 generate $(\langle D_0 \rangle_0, \langle D_0 \rangle_1) \leftarrow \mathcal{F}_{\text{Share}}(D_0)$ and send $\langle D_0 \rangle_1$ to P_1 , and P_1 generate $(\langle D_1 \rangle_0, \langle D_1 \rangle_1) \leftarrow \mathcal{F}_{\text{Share}}(D_1)$ and send $\langle D_1 \rangle_0$ to P_0 . P_0 compute $\langle D \rangle_0 = \langle D_0 \rangle_0 + \langle D_1 \rangle_0$ and P_1 compute $\langle D \rangle_1 = \langle D_0 \rangle_1 + \langle D_1 \rangle_1$ locally.

Specifically, as shown in Fig. 1 (a), FssNN works as follows: parties P_0 and P_1 hold the secret shares of training datasets $\langle D \rangle_0$ and $\langle D \rangle_1$ respectively, and initialize $\langle \mathbf{W} \rangle_0$ and $\langle \mathbf{W} \rangle_1$ to be the all 0s locally. Then, for $b = 0, 1$, P_b randomly selects a training sample $(\langle \mathbf{x}_i \rangle_b, \langle y_i \rangle_b)$ for $\langle D \rangle_b$, and engages in a secure two-party SGD protocol (2PC-SGD) with P_{1-b} to update $\langle \mathbf{W} \rangle$ interactively, which involves two steps: ① forward propagation and ② backward propagation. During the forward propagation and the backward propagation, we need to securely compute linear layers (denoted by green solid line boxes) and non-linear layers (denoted by blue dashed line boxes). Finally, P_0 and P_1 select a new sample randomly and repeat the above process until the samples are used up, and finally P_b gets $\langle \mathbf{W} \rangle_b$ and sends $\langle \mathbf{W} \rangle_b$ to P_{1-b} to reconstruct $\mathbf{W}$.

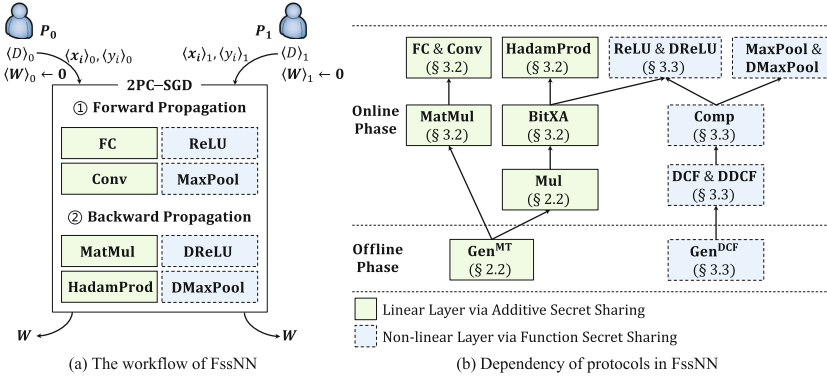


Fig. 1. The high level overview of FssNN

To reduce the communication rounds and communication costs required to compute linear layers and non-linear layers, we propose a hybrid method combining additive secret sharing (SS) and function secret sharing (FSS) and adopt the offline-online paradigm to compute linear layers in one round of online communication and compute non-linear layers in a constant round of online communication. Figure 1 (b) shows our secure linear layer protocols (denoted by blue dashed line boxes) and secure non-linear layer protocols (denoted by blue dashed line boxes) in the offline-online paradigm.

For linear layers, by leveraging a SS-based secure two-party computation protocol, we generate multiplication triples (Gen^{MT}) in the offline phase and utilize these multiplication triples to compute linear layer functions such as MatMul , Conv , FC and HadamProd in one round of communication in the online phase. Among them, we propose a protocol, BitXA , that supports direct multiplication of a bit and an integer to reduce online communication costs, and extend it to

HadamProd through vectorization techniques. **For non-linear layers**, we propose a key-reduced DCF scheme with compact additive construction and design a distributed DCF key generation scheme based on an MPC-friendly pseudo-random generator (PRG). In the offline phase, we design a distributed DCF key generation scheme (Gen^{DCF}) for the proposed key-reduced DCF to generate the DCF key rather than relying on a trusted party, and utilize the DCF key to compute non-linear layer functions such as ReLU, DReLU, MaxPool and DMaxPool with a constant-round online communication.

3.2 Construction of Secure Linear Layers

In this section, we present the detailed construction of linear layer functions, i.e., matrix multiplication MatMul and Hadamard product HadamProd .

Secure Matrix Multiplication. A fully-connected layer is exactly a matrix multiplication, thus secure fully-connected layer (FC) can be implemented directly using Π_{MatMul} . Likewise, convolutional layer can also be expressed as a matrix multiplication using an unrolling technique (see Fig. 3 in [22]), so secure convolution layer (Conv) can also be implemented using Π_{MatMul} .

By leveraging the vectorization technique in [20], secure scalar multiplication (Mul) introduced in Sect. 2.2 can be easily extended to secure matrix multiplication (MatMul) where the multiplication triples are replaced by matrix multiplication triples. Given secret shares of matrices $\langle \mathbf{X} \rangle, \langle \mathbf{Y} \rangle$ held by P_0 and P_1 where $\mathbf{X} \in \mathbb{Z}_{2^n}^{m_1 \times m_2}, \mathbf{Y} \in \mathbb{Z}_{2^n}^{m_2 \times m_3}$, functionality $\mathcal{F}_{\text{MatMul}}$ computes $\langle \mathbf{Z} \rangle$ such that $\mathbf{Z} = \mathbf{X} \times \mathbf{Y}$. Π_{MatMul} realizes the functionality $\mathcal{F}_{\text{MatMul}}$ as follows: 1) In offline phase, P_b samples $\langle \mathbf{U} \rangle_b, \langle \mathbf{V} \rangle_b$ randomly, and then parties invoke $\text{Gen}^{\text{MT}}(\langle \mathbf{U} \rangle, \langle \mathbf{V} \rangle)$ to generate matrix multiplication triples $(\langle \mathbf{U} \rangle, \langle \mathbf{V} \rangle, \langle \mathbf{UV} \rangle)$. 2) In the online phase, parties open $\mathbf{X} - \mathbf{U}$ and $\mathbf{Y} - \mathbf{V}$, and then P_b computes $\langle \mathbf{Z} \rangle_b = b \cdot (\mathbf{X} - \mathbf{U}) \times (\mathbf{Y} - \mathbf{V}) + (\mathbf{X} - \mathbf{U}) \times \langle \mathbf{V} \rangle_b + \langle \mathbf{U} \rangle_b \times (\mathbf{Y} - \mathbf{V}) + \langle \mathbf{UV} \rangle_b$ locally. This requires an online communication of $(m_1 m_2 + m_2 m_3) \cdot n$ bits in 1 round.

Secure Hadamard Product. Neural network training makes extensive use of Hadamard product in backpropagation. Observe that Hadamard product operations (denoted by $\odot$) have a specific structure that can be leveraged to reduce communication costs: when computing $\mathbf{X} \odot \mathbf{Y}$ where $\mathbf{X} \in \mathbb{Z}_{2^n}^{m_1 \times m_2}, \mathbf{Y} \in \mathbb{Z}_2^{m_2 \times m_3}$, the each element x in $\mathbf{X}$ is a n -bit integer and the each element y in $\mathbf{Y}$ is a bit. However, the arithmetic share $\langle x \rangle^A$ can not be directly multiplied by the Boolean share $\langle y \rangle^B$ since they are calculated with different moduli. Existing works first convert $\langle y \rangle^B$ to the arithmetic share $\langle y \rangle^A$ and then perform the multiplication of $\langle x \rangle^A$ and $\langle y \rangle^A$, incurring an online communication of $2m_1 m_2 n$ bits in 2 rounds.

In order to reduce online communication costs, we propose an online-efficient Hadamard product protocol to support the direct computation of $\mathbf{X} \odot \mathbf{Y}$ by moving the share conversion into the offline phase, which requires an online communication of $m_1 m_2 (n + 1)$ bits in 1 round. We present a scalar protocol, Π_{BitXA} , to support the product of $\langle x \rangle^A$ and $\langle y \rangle^B$, which can be easily extended to the vector protocol $\Pi_{\text{HadamProd}}$ through the vectorization technique.

Given the arithmetic share $\langle x \rangle^A$ and Boolean share $\langle y \rangle^B$, functionality $\mathcal{F}_{\text{BitXA}}$ generates $\langle z \rangle^A$ with $z = x \cdot y$ and the protocol is shown in Algorithm 1. Π_{BitXA} needs an online communication of $n + 1$ bits per party in 1 round.

Algorithm 1. BitXA: $\Pi_{\text{BitXA}}(\langle y \rangle^B, \langle x \rangle^A)$

Input: P_0 and P_1 hold $\langle x \rangle^A$ and $\langle y \rangle^B$.

Output: P_b gets $\langle z \rangle_b^A$ such that $\langle z \rangle_0^A + \langle z \rangle_1^A = (\langle x \rangle_0^A + \langle x \rangle_1^A) \cdot (\langle y \rangle_0^B \oplus \langle y \rangle_1^B)$.

• **Offline Phase**

- 1: P_b samples $\langle \hat{\delta}_y \rangle_b^B \in_R \mathbb{Z}_2$, and then P_0 and P_1 convert it to arithmetic share:
 - (1) P_0 lets $\langle e \rangle_0^A = \langle \delta_y \rangle_0^B$, $\langle f \rangle_0^A = 0$, and P_1 lets $\langle e \rangle_1^A = 0$, $\langle f \rangle_1^A = \langle \delta_y \rangle_1^B$.
 - (2) P_0 and P_1 compute $(\langle ef \rangle_0^A, \langle ef \rangle_1^A) \leftarrow \Pi_{\text{Mul}}(\langle e \rangle^A, \langle f \rangle^A)$.
 - (3) P_b computes $\langle \delta_y \rangle_b^A = \langle e \rangle_b^A + \langle f \rangle_b^A - 2 \cdot \langle ef \rangle_b^A$ locally.
- 2: P_b samples $\langle \delta_x \rangle_b^A \in_R \mathbb{Z}_{2^n}$, and then parties compute $(\langle \delta_z \rangle_0^A, \langle \delta_z \rangle_1^A) \leftarrow \Pi_{\text{Mul}}(\langle \delta_x \rangle^A, \langle \delta_y \rangle^A)$ such that $\delta_z = \delta_x \cdot \delta_y$.
- 3: P_0 and P_1 hold $\langle \delta_x \rangle^A, \langle \delta_y \rangle^A, \langle \delta_z \rangle^A$ and $\langle \hat{\delta}_y \rangle^B$.

• **Online Phase**

- 1: P_b locally computes $\langle x \rangle_b^A + \langle \delta_x \rangle_b^A, \langle y \rangle_b^B \oplus \langle \hat{\delta}_y \rangle_b^B$, and sends to P_{1-b} .
 - 2: P_b reconstructs $\Delta_x = x + \delta_x, \Delta_y = y \oplus \hat{\delta}_y$, and sets $\Delta'_y = \Delta_y$ where $\Delta'_y \in \mathbb{Z}_{2^n}$.
 - 3: P_b computes locally $\langle z \rangle_b^A = b \cdot \Delta'_y \cdot \Delta_x + \langle \delta_y \rangle_b^A \cdot \Delta_x - 2 \cdot \Delta'_y \cdot \Delta_x \cdot \langle \delta_y \rangle_b^A - \Delta'_y \cdot \langle \delta_x \rangle_b^A - \langle \delta_z \rangle_b^A + 2 \cdot \Delta'_y \cdot \langle \delta_z \rangle_b^A$.
-

Security Analysis. Theorem 1 captures the security of Π_{BitXA} , and the proof is in the full version of this paper [27]. The security of HadamProd follows in the $\mathcal{F}_{\text{BitXA}}$ -hybrid model.

Theorem 1. *In the $\mathcal{F}_{\text{Mul}}$ -hybrid model, Π_{BitXA} securely computes the functionality $\mathcal{F}_{\text{BitXA}}$ in the presence of semi-honest adversaries.*

3.3 Construction of Secure Non-linear Layers

In this section, we present the construction of non-linear layer functions (i.e., DReLU and ReLU) by using a key-reduced distributed comparison function (DCF) scheme with compact additive construction. To replace the trusted party in the offline phase, we propose a DCF key generation scheme based on MPC-friendly pseudorandom generators, which supports a larger input domain. In this paper, we directly use the secure maxpool algorithm proposed in [22] (see algorithm 7 in [22]) and its derivative, but utilize our proposed DCF construction.

Secure DReLU and ReLU. In neural network, ReLU is one of the most popular activation functions in neural network training. For a signed value x , ReLU is define as $\max(0, x)$ and its derivative is defined as $\mathbf{1}\{x \geq 0\}$. Given an arithmetic share $\langle x \rangle$, the functionality $\mathcal{F}_{\text{ReLU}}$ outputs the arithmetic share of $\max(0, x)$, and the functionality $\mathcal{F}_{\text{DReLU}}$ outputs the Boolean share of $\mathbf{1}\{x \geq 0\}$. It can be seen that $\text{ReLU}(x) = x \cdot \text{DReLU}(x)$. In this section, by leveraging the

DDCF scheme constructed by using the proposed DCF, we first design a signed integer comparison gate scheme to implement DReLU , and then compute ReLU using $\Pi_{\text{ReLU}}(\langle x \rangle^A) = \Pi_{\text{BitXA}}(\langle x \rangle^A, \langle \Pi_{\text{DReLU}}(\langle x \rangle^A) \rangle^B)$.

Secure. DReLU To implement $\mathcal{F}_{\text{DReLU}}$, we first propose signed integer comparison gate Comp in Algorithm 2 which is derived from [2]. In the Algorithm 2, $(\text{Gen}_{n-1}^{\text{DDCF}}, \text{Eval}_{n-1}^{\text{DDCF}})$ is used to evaluate $f_{\alpha^{(n-1)}, \beta_0, \beta_1}^<(x)$ that outputs β_0 for $0 \leq x < \alpha^{(n-1)}$ and β_1 for $x \geq \alpha^{(n-1)}$, and its detailed construction is shown in the Appendix A. Comp requires 1 call of DDCF and the key sizes are $(\lceil n-1-\log \lambda \rceil)(\lambda+3)+2\lambda$ bits per party where λ is the security parameter.

Base on Comp , Π_{DReLU} is proposed to compute $\mathbf{1}\{x \geq 0\}$, and this protocol is in Algorithm 3 where a trusted party is used to precompute FSS keys of Comp . The trusted party can be instantiated via using our proposed distributed DCF key generation scheme (Algorithm 5). Π_{DReLU} requires 1 call of Comp in the offline phase, and requires requires 1 round with n bits in the online phase.

Secure. ReLU $\mathcal{F}_{\text{ReLU}}$ is implemented by computing $\Pi_{\text{ReLU}}(\langle x \rangle^A) = \Pi_{\text{BitXA}}(\langle x \rangle^A, \langle \Pi_{\text{DReLU}}(\langle x \rangle^A) \rangle^B)$, which needs the same key sizes as Π_{DReLU} and requires 1 round with $2n+1$ bits in the online phase.

Algorithm 2. Signed Integer Comparison Gate $\text{Comp} : (\text{Gen}_n^{\text{Comp}}, \text{Eval}_n^{\text{Comp}})$

- $\text{Gen}_n^{\text{Comp}}(1^\lambda, r_1^{\text{in}}, r_2^{\text{in}}, r^{\text{out}})$
 - 1: Let $r = (2^n - (r_1^{\text{in}} - r_2^{\text{in}})) \in \mathbb{Z}_{2^n}$, and $\alpha^{(n-1)} = r_{[0, n-1]}$.
 - 2: $(k_0^{(n-1)}, k_1^{(n-1)}) \leftarrow \text{Gen}_{n-1}^{\text{DDCF}}(1^\lambda, \alpha^{(n-1)}, \beta_0, \beta_1)$, where $\beta_0 = 1 \oplus r_{[n-1]}$, $\beta_1 = r_{[n-1]}$.
 - 3: Sample randoms $r_0, r_1 \in_R \mathbb{Z}_2$, such that $r_0 \oplus r_1 = r^{\text{out}}$.
 - 4: Let $k_b = k_b^{(n-1)} || r_b$.
 - 5: **return** (k_0, k_1) .
 - $\text{Eval}_n^{\text{Comp}}(b, k_b, \hat{x}, \hat{y})$
 - 1: P_b parses $k_b = k_b^{(n-1)} || r_b$, and lets $z = (\hat{x} - \hat{y}) \in \mathbb{Z}_{2^n}$.
 - 2: P_b computes $m_b \leftarrow \text{Eval}_{n-1}^{\text{DDCF}}(b, k_b^{(n-1)}, z^{(n-1)})$, where $z^{(n-1)} = 2^{n-1} - z_{[0, n-1]} - 1$.
 - 3: P_b lets $v_b = (b \cdot z_{[n-1]}) \oplus m_b \oplus r_b$.
 - 4: **return** v_b .
-

Security Analysis. Theorem 2 captures the security of Π_{DReLU} , and the proof is given in the full version of this paper [27]. The security of Π_{ReLU} follows in $(\mathcal{F}_{\text{BitXA}}, \mathcal{F}_{\text{DReLU}})$ -hybrid model.

Theorem 2. *In the $\mathcal{F}_{\text{Gen}_n^{\text{Comp}}}$ -hybrid model, Π_{DReLU} securely computes the functionality $\mathcal{F}_{\text{DReLU}}$ in the presence of semi-honest adversaries.*

Key-Reduced DCF with a Compact Additive Construction. A central building block in FssNN is a distributed comparison function (DCF), which is intensively used in neural network to build activation functions like ReLU (and

Algorithm 3. DReLU : $\Pi_{\text{DReLU}}(\langle x \rangle^A)$ **Input:** P_0 and P_1 hold arithmetic share $\langle x \rangle^A$.**Output:** P_0 and P_1 obtain Boolean share $\langle y \rangle^B$, such that $\langle y \rangle_0^B \oplus \langle y \rangle_1^B = \mathbf{1}\{x \geq 0\}$.**• Offline Phase**1: Compute $(k_0, k_1) \leftarrow \text{Gen}_n^{\text{Comp}}(1^\lambda, r_1^{\text{in}}, r_2^{\text{in}}, r^{\text{out}})$.2: Send k_b , $\langle r_1^{\text{in}} \rangle_b^A$, $\langle r^{\text{out}} \rangle_b^B$ and r_2^{in} to P_b .**• Online Phase**1: P_b computes $\langle x + r_1^{\text{in}} \rangle_b^A = \langle x \rangle_b^A + \langle r_1^{\text{in}} \rangle_b^A$ and sends $\langle x + r_1^{\text{in}} \rangle_b^A$ to P_{1-b} .2: P_b computes $x + r_1^{\text{in}} = \langle x + r_1^{\text{in}} \rangle_0^A + \langle x + r_1^{\text{in}} \rangle_1^A$.3: P_b computes $\langle y \rangle_b^B \leftarrow b \oplus \text{Eval}_n^{\text{Comp}}(b, k_b, x + r_1^{\text{in}}, r_2^{\text{in}}) \oplus \langle r^{\text{out}} \rangle_b^B$ locally.4: **return** $\langle y \rangle_b^B$.

its derivative). We examine the case of $x, \alpha \in \mathbb{Z}_{2^n}$ and $\beta \in \mathbb{Z}_2$ and propose a key-reduced DCF scheme with compact additive construction for comparison function $f_{\alpha, \beta}^<(x)$. Our construction draws inspiration from the distributed point function of [4], and our construction involves the algorithm $(\text{Gen}_n^<, \text{Eval}_n^<)$. The proposed DCF construction has the following two differences compared the state-of-the-art work, called BCG+21 [2]:

- (1) We maintain input group $\mathbb{G}^{\text{in}} = \mathbb{Z}_{2^n}$ but let the output group $\mathbb{G}^{\text{out}} = \mathbb{Z}_2$ rather than $\mathbb{G}^{\text{out}} = \mathbb{Z}_{2^m}$ where m, n are integers, and propose a key-reduced DCF construction by integrating correction words and designing a more compact key generation algorithm.
- (2) We apply the idea of early termination in [4] to reduce the number of actually required correction words, thereby further reducing the DCF key sizes. Therefore, our proposed DCF construction only supports the output group $\mathbb{G}^{\text{out}} = \mathbb{Z}_2$, but it has smaller key sizes than BCG+21 [2] from $n(\lambda+3) + \lambda + 1$ bits to $(\lceil n - \log \lambda \rceil)(\lambda + 3) + 2\lambda$ bits where λ is the security parameter.

Distributed Comparison Function. $(\text{Gen}_n^<, \text{Eval}_n^<)$ are presented by Algorithm 4, where $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2(\lambda+2)}$ be a PRG, and $\parallel$ is a concatenation operator. In Algorithm 4, the number of PRG invocations in $\text{Gen}_n^<$ is $2n$ and the number of PRG invocations in $\text{Eval}_n^<$ is n . The gray parts are the differences between our DCF scheme and the DCF scheme in BCG+21 [2].

Early Termination Optimization. According to the description of the early termination technique in Boyle’s distributed point function scheme [4], if the length of elements in the output group of a point function is short than the length of random string generated for each node, then several outputs can be packed into a single correction word. We can further improve the complexity of $(\text{Gen}_n^<, \text{Eval}_n^<)$ by using the “early termination” optimization, which works as follows: for any node V of depth v in the tree, there are 2^{n-v} leaf nodes in its sub-tree, or 2^{n-v} input elements with a shared prefix that ends at V . If the size of $CW^{(v+1)}$ is at least 2^{n-v} times the output length then the subsequent correction words, especially v_{CW}^L , can be computed and packed into a single $CW^{(v+1)}$

Algorithm 4. Distributed Comparison Function (DCF): $(\text{Gen}_n^<, \text{Eval}_n^<)$

• **Key Generation Algorithm $\text{Gen}_n^<(1^\lambda, \alpha, \beta)$**

- 1: Let $\alpha_1 || \dots || \alpha_n \in \{0, 1\}^n$ be the bit decomposition of α .
- 2: Sample randoms $s_0^{(0)} \in_R \{0, 1\}^\lambda$ and $s_1^{(0)} \in_R \{0, 1\}^\lambda$, and $t_0^{(0)} = 0, t_1^{(0)} = 1$.
- 3: Let $v_0^{(0)} = 0, v_1^{(0)} = 0$.
- 4: **for** $i = 1$ to n **do**
- 5: $s_0^L || v_0^L || t_0^L || s_0^R || v_0^R || t_0^R \leftarrow G(s_0^{(i-1)})$, and $s_1^L || v_1^L || t_1^L || s_1^R || v_1^R || t_1^R \leftarrow G(s_1^{(i-1)})$.
- 6: **if** $\alpha_i = 0$ **then** $\text{Keep} \leftarrow L, \text{Lose} \leftarrow R$.
- 7: **else** $\text{Keep} \leftarrow R, \text{Lose} \leftarrow L$.
- 8: $s_{CW}^{\text{Lose}} \leftarrow s_0^{\text{Lose}} \oplus s_1^{\text{Lose}}$, and $v_{CW}^L \leftarrow v_0^L \oplus v_1^L \oplus (\alpha_i \cdot \beta)$.
- 9: $t_{CW}^L \leftarrow t_0^L \oplus t_1^L \oplus \alpha_i \oplus 1$, and $t_{CW}^R \leftarrow t_0^R \oplus t_1^R \oplus \alpha_i$.
- 10: $CW^{(i)} = s_{CW} || v_{CW}^L || t_{CW}^L || t_{CW}^R$.
- 11: $s_0^{(i)} = s_0^{\text{Keep}} \oplus t_0^{(i-1)} \cdot s_{CW}, s_1^{(i)} = s_1^{\text{Keep}} \oplus t_1^{(i-1)} \cdot s_{CW}$.
- 12: $t_0^{(i)} = t_0^{\text{Keep}} \oplus t_0^{(i-1)} \cdot t_{CW}^{\text{Keep}}, t_1^{(i)} = t_1^{\text{Keep}} \oplus t_1^{(i-1)} \cdot t_{CW}^{\text{Keep}}$.
- 13: **end for**
- 14: Let $k_b = s_b^{(0)} || CW^{(1)} || \dots || CW^{(n)}$.
- 15: **return** (k_0, k_1) .

• **Evaluation Algorithm $\text{Eval}_n^<(b, k_b, x)$**

- 1: Parse $k_b = s^{(0)} || CW^{(1)} || \dots || CW^{(n)}$, and let $x = x_1 || \dots || x_n, v = 0$, and $t^{(0)} = b$.
- 2: **for** $i = 1$ to n **do**
- 3: Parse $CW^{(i)} = s_{CW} || v_{CW}^L || t_{CW}^L || t_{CW}^R$, and compute $G(s^{(i-1)}) = \hat{s}^L || \hat{v}^L || \hat{t}^L || \hat{s}^R || \hat{v}^R || \hat{t}^R$.
- 4: $\tau^{(i)} \leftarrow [\hat{s}^L || \hat{v}^L || \hat{t}^L || \hat{s}^R || \hat{v}^R || \hat{t}^R] \oplus (t^{(i-1)} \cdot [s_{CW} || v_{CW}^L || t_{CW}^L || s_{CW} || v_{CW}^L || t_{CW}^R])$.
- 5: Parse $\tau^{(i)} = s^L || v^L || t^L || s^R || v^R || t^R$.
- 6: **if** $x_i = 0$ **then** $v \leftarrow v \oplus v^L, s^{(i)} \leftarrow s^L, t^{(i)} \leftarrow t^L$.
- 7: **else** $s^{(i)} \leftarrow s^R, t^{(i)} \leftarrow t^R$.
- 8: **end for**
- 9: **return** v

instead of involving all subsequent correction words. In this case, $CW^{(v+1)}$ will be a sequence of $v_{CW}^{L, \hat{\alpha}} = v_{CW}^{L, v+1} \oplus \dots \oplus v_{CW}^{L, n}$ where $\hat{\alpha} \in \mathbb{Z}_{2^{n-v}}$ and $v_{CW}^{L, i}$ is the last $n - v$ values in all v_{CW}^L s (i.e., $v_{CW}^{L, i}$ for $i = v+1, \dots, n$) where v_{CW}^L is defined in the line 11 in Algorithm 4. The sequence will output β in the location specified by $\alpha_{[v+1, n]} = \alpha_{v+1} || \dots || \alpha_n$, and 0 in every other location.

In this paper, we let $v = \lceil n - \log \lambda \rceil$ to satisfy the above conditions. Therefore, there are only v correction words of size $\lambda + 3$, plus a $CW^{(v+1)}$ of size $2^{n-v} = \lambda$, thus the total DCF key sizes are $(\lceil n - \log \lambda \rceil)(\lambda + 3) + 2\lambda$ bits.

Security Analysis. Theorem 3 captures the correctness and security of the DCF construction, and its full proof can be found in the full paper [27].

Theorem 3. (*Correctness and Security*) The scheme $(\text{Gen}_n^<, \text{Eval}_n^<)$ from Algorithm 4 is a DCF for the family of comparison functions $f_{\alpha, \beta}^<(x) : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_2$ with key sizes $(\lceil n - \log \lambda \rceil)(\lambda + 3) + 2\lambda$ bits, where λ is the security parameter.

Distributed DCF Key Generation via MPC-friendly PRG. A trusted party is required to execute the procedure $\text{Gen}_n^{\text{Comp}}$ for **Comp** in Algorithm 3, and since **Comp** is constructed based on the proposed DCF, the DCF key needs to be computed indeed. To realize the functionality $\mathcal{F}_{\text{GenDCF}}$, we present Π_{GenDCF} based on a secure two-party PRG to generate DCF key, and this protocol is in Algorithm 5, where $\mathcal{F}_{\text{SecPRG}}$ can be realized by the MPC-friendly PRG [9] and $\mathcal{F}_{2\text{PC}}$ can be instantiate via using a secure two-party protocol based on secret sharing [7, 8, 21]. Note that Algorithm 5 is naturally extended to the case of using the early termination optimization.

The scheme in BCG+21 [2] (Fig. 9 in BCG+21 [2]) needs $O(2^n)$ invocations of PRG and is restricted to moderate domain sizes. By comparison, Π_{GenDCF} only requires $O(n)$ invocations of PRG and can be used with larger domain sizes. Although the Appendix A.2 in [2] also mentions a distributed DCF key generation scheme via a generic 2PC with $O(n)$ evaluations of PRG, it does not give a specific construction. More importantly, the scheme mentioned in BCG+21 [2] is only applicable to the DCF construction proposed in BCG+21 [2] and cannot be used to generate the key of our proposed DCF construction, because our proposed DCF construction is essentially different from the DCF construction of BCG+21 [2].

Algorithm 5. $\Pi_{\text{GenDCF}}(1^\lambda, b, \{\langle \alpha_i \rangle_b^B\}_{i=1, \dots, n}, \langle \beta \rangle_b^B)$

Input: Party index b , and P_b holds $\{\langle \alpha_i \rangle_b^B\}_{i=1, \dots, n}$ and $\langle \beta \rangle_b^B$.

Output: P_b gets DCF key k_b .

- 1: P_b samples randoms $s_b^{(0)} \in_R \{0, 1\}^\lambda, t_b^{(0)} = b$.
- 2: P_b invokes $\Pi_{\text{Share}}(s_b^{(0)}), \Pi_{\text{Share}}(t_b^{(0)})$ to generate secret shares of $s_b^{(0)}$ and $t_b^{(0)}$, then P_b obtains $\langle s_0^{(0)} \rangle_b^B, \langle t_0^{(0)} \rangle_b^B, \langle s_1^{(0)} \rangle_b^B, \langle t_1^{(0)} \rangle_b^B$.
- 3: **for** $i = 1$ to n **do**
- 4: P_0 and P_1 engage in a secure two-party PRG to compute (for $j \in \{0, 1\}$):

$$(\langle G(s_j^{(i-1)}) \rangle_0^B, \langle G(s_j^{(i-1)}) \rangle_1^B) \leftarrow \mathcal{F}_{\text{SecPRG}}(\langle s_j^{(i-1)} \rangle_0^B, \langle s_j^{(i-1)} \rangle_1^B)$$

$$\text{where } \langle G(s_j^{(i-1)}) \rangle_0^B \oplus \langle G(s_j^{(i-1)}) \rangle_1^B = s_j^{L, i-1} || v_j^{L, i-1} || t_j^{L, i-1} || s_j^{L, i-1} || v_j^{L, i-1} || t_j^{L, i-1}.$$

- 5: P_0 and P_1 make access to $\mathcal{F}_{2\text{PC}}$ to compute:

$$\begin{aligned} (s_{CW}^L, v_{CW}^L) &\leftarrow \begin{cases} (s_0^{R, i-1} \oplus s_1^{R, i-1}, v_0^{L, i-1} \oplus v_1^{L, i-1}) & \alpha_i = 0 \\ (s_0^{L, i-1} \oplus s_1^{L, i-1}, v_0^{L, i-1} \oplus v_1^{L, i-1} \oplus \beta) & \alpha_i = 1 \end{cases} \\ (t_{CW}^L, t_{CW}^R) &\leftarrow \begin{cases} (t_0^{L, i-1} \oplus t_1^{L, i-1} \oplus 1, t_0^{R, i-1} \oplus t_1^{R, i-1}) & \alpha_i = 0 \\ (t_0^{L, i-1} \oplus t_1^{L, i-1}, t_0^{R, i-1} \oplus t_1^{R, i-1} \oplus 1) & \alpha_i = 1 \end{cases} \end{aligned}$$

P_0 and P_1 obtain $s_{CW}; v_{CW}^L; t_{CW}^L, t_{CW}^R$.

- 6: P_b computes $CW^{(i)} = s_{CW} || v_{CW}^L || t_{CW}^L || t_{CW}^R$ locally.
- 7: P_0 and P_1 make access to $\mathcal{F}_{2\text{PC}}$ to compute:

$$s_b^{(i)} \leftarrow \begin{cases} s_b^{L, i-1} \oplus t_b^{(i-1)} \cdot s_{CW} & \alpha_i = 0 \\ s_b^{R, i-1} \oplus t_b^{(i-1)} \cdot s_{CW} & \alpha_i = 1 \end{cases}, \quad t_b^{(i)} \leftarrow \begin{cases} t_b^{L, i-1} \oplus t_b^{(i-1)} \cdot t_{CW}^L & \alpha_i = 0 \\ t_b^{R, i-1} \oplus t_b^{(i-1)} \cdot t_{CW}^R & \alpha_i = 1 \end{cases}$$

P_0 and P_1 obtain $(\langle s_b^{(i)} \rangle_0^B, \langle t_b^{(i)} \rangle_0^B), (\langle s_b^{(i)} \rangle_1^B, \langle t_b^{(i)} \rangle_1^B)$ respectively, where $b = 0, 1$.

8: **end for**

- 9: P_b lets $k_b \leftarrow s_b^{(0)} || CW^{(1)} || \dots || CW^{(n)}$.
-

4 Theoretical Analysis and Experiment

In the section, we present the theoretical analysis of the communication and computation complexity in Sect. 4.1, and show the experiment results in Sect. 4.2.

4.1 Theoretical Analysis

Online Round and Communication Complexity. The online rounds and communication costs of each neural network operation in ABY2.0 [21], AriaNN [22] and FssNN (Ours) are presented in Table 2. The function $\text{MatMul}_{m_1, m_2, m_3}$ denotes a matrix multiplication of dimension $m_1 \times m_2$ with $m_2 \times m_3$, and $\text{HadamProd}_{m_1, m_2}$ denotes a Hadamard product of dimension $m_1 \times m_2$. ReLU_{m_1, m_2} and DReLU_{m_1, m_2} denotes ReLU and its derivative over a $m_1 \times m_2$ matrix, and $\text{MaxPool}_{m, k, s}$ denotes maxpool with input the $m \times m$ where k stands for the kernel size and s stands for the stride. All communication is measured for n -bit inputs and missing entries mean that data was not available.

For round complexity, all neural network operators are computed with constant online communication rounds in AriaNN and FssNN, while linear functions (i.e., MatMul and HadamProd) computation requires 1 round and non-linear (i.e., DReLU , ReLU and MaxPool) computation requires $O(\log n)$ rounds in ABY2.0. For online communication costs, FssNN achieves lower online communication costs in HadamProd and ReLU than AriaNN due to our communication efficient protocol Π_{BitXA} .

Table 2. Online rounds and communication of ABY2.0 [21], AriaNN [22] and FssNN (Ours)

Operators	Rounds			Communication		
	ABY2.0	AriaNN	FssNN	ABY2.0	AriaNN	FssNN
$\text{MatMul}_{m_1, m_2, m_3}$	1	1	1	$m_1 m_3 n$	$(m_1 m_2 + m_2 m_3) n$	$(m_1 m_2 + m_2 m_3) n$
$\text{HadamProd}_{m_1, m_2}$	1	1	1	$2 m_1 m_2 n$	$2 m_1 m_2 n$	$m_1 m_2 (n + 1)$
DReLU_{m_1, m_2}	$1 + \log n$	1	1	$\sim 3 m_1 m_2 n$	$m_1 m_2 n$	$m_1 m_2 n$
ReLU_{m_1, m_2}	$2 + \log n$	2	2	$\sim 5 m_1 m_2 n$	$3 m_1 m_2 n$	$m_1 m_2 (2n + 1)$
$\text{MaxPool}_{m, k, s}$	-	3	3	-	$(\frac{m-k}{s} + 1)^2 (k^4 + 2) n$	$(\frac{m-k}{s} + 1)^2 (k^4 + 2) n$

Offline Communication and Computation Complexity. In the online phase, ABY2.0 [21], AriaNN [22] and FssNN (Ours) all have an order of magnitude of online computation complexity since they all adopt the offline-online paradigm. In the offline phase, FssNN uses the same multiplication triples generation scheme as ABY2.0 and AriaNN. However, FssNN needs to generate correlated randomness (i.e., DCF key) to compute DReLU , ReLU and MaxPool , while ABY2.0 requires smaller correlated randomness and it can be generated more efficiently using 2PC-based offline phase (but leads to $4 - 5 \times$ more rounds and $3 - 6 \times$ more communication of online communication [2]) and AriaNN relies on

a trusted party to correlated randomness (but leads to stronger assumptions). Therefore, FssNN requires more communication and computation in the offline phase, but has less online communication compared with ABY2.0 and does not rely on the trusted leader compared with AriaNN.

DCF Key Sizes. The communication criteria of DCF construction is the sizes of key k_b (i.e., the output of $\text{Gen}_n^<$), so the DCF key sizes in BCG+21 [2], AriaNN [22] and FssNN are shown in Table 3. It is clear that the DCF key sizes of FssNN is the smallest, which decreases the size of DCF keys by about 17.9%.

Table 3. The DCF key sizes in BCG+21 [2], AriaNN [22] and FssNN (Ours)

Para. (bits)	BCG+21 [2]	AriaNN [22]	FssNN (Ours)
(n, λ)	$n(\lambda + 3) + \lambda + 1$	$n(\lambda + 2n + 4) + \lambda + 2n$	$(\lceil n - \log \lambda \rceil)(\lambda + 3) + 2\lambda$
(32, 127)	4424	6431	3634

4.2 Experiment

In this section, we present the implement of FssNN and the detailed experiment results and analysis. We implement FssNN in Python and run the experiments on Aliyun ESC using ecs.hfr7.xlarge machines with 32 cores and 256 GB of CPU RAM in a LAN setting. In order to simplify comparison with existing works, we follow a setup very close to AriaNN [22] and use same neural network models and datasets. AriaNN [22] is the state-of-the-art secure neural network training and inference framework based on function secret sharing, and outperform other works such as FALCON [25] and ABY2.0 [21].

Evaluations for Secure Layer Functions. First, we present the offline and online communication costs of linear layer functions (i.e., MatMul and HadamProd) and non-linear layer functions (i.e., DReLU, ReLU and MaxPool) in Table 4.

Table 4. Offline and online communication of neural network operators in AriaNN [22] and FssNN where (784, 128, 10), (128, 128) and (24, 2, 2) are typical parameters.

Operators	Input Sizes	Offline Comm. (MB)		Online Comm. (MB)	
		AriaNN	FssNN (Ours)	AriaNN	FssNN (Ours)
MatMul $_{m_1, m_2, m_3}$	(784, 128, 10)	0.842	0.842	0.775	0.775
HadamProd $_{m_1, m_2}$	(128, 128)	0.381	0.272 (↓ 28.6%)	0.251	0.142 (↓ 43.4%)
DReLU $_{m_1, m_2}$	(128, 128)	14.377	10.314 (↓ 28.3%)	0.126	0.126
ReLU $_{m_1, m_2}$	(128, 128)	14.758	10.586 (↓ 28.3%)	0.377	0.268 (↓ 28.9%)
MaxPool $_{m, k, s}$	(24, 2, 2)	0.399	0.292 (↓ 26.8%)	0.020	0.020

For linear layer functions, compared with AriaNN, the offline and online communication costs of **HadamProd** decreases by 28.6% and 43.4% respectively. For non-linear layer functions, we improve the offline communication costs by 26.8%–28.3% over AriaNN due to the proposed key-reduced DCF in FssNN. The online communication improvement of ReLU is attributed to our communication efficient Π_{BitXA} .

Evaluations for Secure Neural Network. We benchmark secure training and inference on MNIST (60,000 training samples and 10,000 test samples) and evaluate following 3 neural networks: (1) a 3-layer fully-connected network (FCNN), (2) a 4-layer convolutional neural network (CNN) and (3) a 4-layer LeNet network (LeNet). It should be noted that FssNN, like AriaNN [22], can support to more machine learning tasks and datasets, such as models AlexNet and VGG16 in datasets CIFAR10 and Tiny Imagenet.

Time, communication and accuracy of secure training and inference in the LAN setting are presented in Table 5, where accuracy of plaintext training and inference are also reported for comparison. The time and communication for secure training are given in hours and GB per epoch, and secure inference is evaluated over pre-trained neural network models and the total time and communication are reported.

It is observed that accuracy of secure training and inference are a little lower than their plaintext counterparts, but the gap between them isn’t significant.

Table 5. Time, communication and accuracy of secure training and inference in FssNN

FssNN	Model	Epochs	Time (h)	Comm. (GB)	Private Acc.	Plaintext Acc.
Training	FCNN	15	0.23	27.35	98.00%	98.04%
	CNN	10	2.24	439.78	98.60%	98.73%
	LeNet	10	3.46	648.83	98.93%	99.03%
Inference	FCNN	–	0.01	2.14	98.15%	98.17%
	CNN	–	0.30	72.84	98.95%	99.02%
	LeNet	–	0.56	107.63	99.22%	99.27%

Compare with AriaNN [22]. The total communication and time for secure training and inference are presented in Table 6. Compared with AriaNN [22], the communication for training declined by 24.3% – 25.4% and the communication for inference decreased by 22.9% – 26.4%. This is attributed to our online-efficient protocol Π_{BitXA} , and key-reduced DCF scheme which improves the communication efficiency of protocols Π_{DReLU} , Π_{ReLU} and Π_{MaxPool} .

Table 6. The communication and time for secure training and inference

	Model	Training		Inference	
		Comm. (GB)	Time (h)	Comm. (GB)	Time (h)
AriaNN [22]	FCNN	36.11	0.28	2.84	0.02
	CNN	589.91	2.24	94.49	0.37
	LeNet	869.75	3.50	146.24	0.56
FssNN(Ours)	FCNN	27.35 (↓ 24.3%)	0.23	2.14 (↓ 24.8%)	0.02
	CNN	439.78 (↓ 25.4%)	2.24	72.84 (↓ 22.9%)	0.30
	LeNet	648.83 (↓ 25.4%)	3.46	107.63 (↓ 26.4%)	0.56

5 Conclusion

Privacy-preserving neural network based on secure multiparty computation has emerged as a flourishing research area in the past few years, but its practicality is greatly limited due to the low efficiency caused by massive communication costs and a deep dependence on a trusted party. In this work, we proposed a communication-efficient secure neural network framework, FssNN, to enable practical training and inference. Theoretical analysis and experiment show the practical performance of FssNN, and the substantial performance advantage over existing works. Compared with the state-of-the-art solution AriaNN, the communication costs of secure training and inference are decreased by 25.4% and 26.4% respectively. More attempts might be made to construct actively secure protocols to defend against a malicious adversary.

Acknowledgements. We sincerely thank the anonymous reviewers of ProvSec 2024 for their valuable comments. The work is supported by Shenzhen Science and Technology Major Project (KJZD20230923114908017), National Natural Science Foundation of China (62272131), and Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies (2022B1212010005).

A Dual Distributed Comparison Function

Dual DCF (DDCF) is a variant of DCF, is defined as:

$$f_{\alpha, \beta_0, \beta_1}^<(x) = \begin{cases} \beta_0 & x < \alpha \\ \beta_1 & \text{else} \end{cases} \quad (1)$$

where $x, \alpha \in \mathbb{Z}_{2^n}$, $\beta_0, \beta_1 \in \mathbb{Z}_2$, and $\beta_0 \neq \beta_1$.

We present a DDCF scheme based on the DCF scheme, its detailed construction is shown in Algorithm 6. Compared with [2], our DDCF construction is slightly modified: the output group is constrained to be $\mathbb{Z}_2$, and more importantly, our DDCF construction relies on the proposed key-reduced DCF (i.e., $(\text{Gen}_n^<, \text{Eval}_n^<)$, § 3.3). Our DDCF construction requires 1 call of DCF, and the key sizes are $(\lceil n - \log \lambda \rceil)(\lambda + 3) + 2\lambda$ bits where λ is the security parameter.

Algorithm 6. DDCF: $(\text{Gen}_n^{\text{DDCF}}, \text{Eval}_n^{\text{DDCF}})$

-
- $\text{Gen}_n^{\text{DDCF}}(1^\lambda, \alpha, \beta_0, \beta_1)$
 - 1: Compute $(k_0^{(n)}, k_1^{(n)}) \leftarrow \text{Gen}_n^<(1^\lambda, \alpha, \beta_0 \oplus \beta_1)$.
 - 2: Sample $r_0, r_1 \in_R \{0, 1\}$, such that $r_0 \oplus r_1 = \beta_1$.
 - 3: Let $k_b = k_b^{(n)} || r_b$ for $b \in \{0, 1\}$.
 - 4: **return** (k_0, k_1) .
 - $\text{Eval}_n^{\text{DDCF}}(b, k_b, x)$
 - 1: Parse $k_b = k_b^{(n)} || r_b$, and compute $y_b^{(n-1)} \leftarrow \text{Eval}_n^<(b, k_b^{(n)}, x)$.
 - 2: Let $v_b = y_b^{(n-1)} \oplus r_b$.
 - 3: **return** v_b .
-

References

1. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: Feigenbaum, J. (ed.) CRYPTO 1991. LNCS, vol. 576, pp. 420–432. Springer, Heidelberg (1992). https://doi.org/10.1007/3-540-46766-1_34
2. Boyle, E., et al.: Function secret sharing for mixed-mode and fixed-point secure computation. In: Canteaut, A., Standaert, F.-X. (eds.) EUROCRYPT 2021. LNCS, vol. 12697, pp. 871–900. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-77886-6_30
3. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015. LNCS, vol. 9057, pp. 337–367. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46803-6_12
4. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing: improvements and extensions. In: 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 1292–1303. ACM, Vienna (2016). <https://doi.org/10.1145/2976749.2978429>
5. Boyle, E., Gilboa, N., Ishai, Y.: Secure computation with preprocessing via function secret sharing. In: Hofheinz, D., Rosen, A. (eds.) TCC 2019. LNCS, vol. 11891, pp. 341–371. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-36030-6_14
6. Chaudhari, H., Rachuri, R., Suresh, A.: Trident: efficient 4PC framework for privacy preserving machine learning. In: Network and Distributed System Security Symposium (2020). <https://doi.org/10.14722/ndss.2020.23005>
7. Damgård, I., Pastro, V., Smart, N., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 643–662. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-32009-5_38
8. Demmler, D., Schneider, T., Zohner, M.: ABY - a framework for efficient mixed-protocol secure two-party computation. In: Network and Distributed System Security Symposium (2015). <https://doi.org/10.14722/ndss.2015.23113>
9. Dinur, I., et al.: MPC-friendly symmetric cryptography from alternating moduli: candidates, protocols, and applications. In: Malkin, T., Peikert, C. (eds.) CRYPTO 2021. LNCS, vol. 12828, pp. 517–547. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-84259-8_18
10. Doerner, J., Shelat, A.: Scaling ORAM for secure computation. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp. 523–535. ACM, Dallas (2017). <https://doi.org/10.1145/3133956.3133967>

11. Feng, D., Yang, K.: Concretely efficient secure multi-party computation protocols: survey and more. In: Security and Safety, p. 2021001 (2022). <https://doi.org/10.1051/sands/2021001>
12. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: 41st Annual ACM Symposium on Theory of Computing, pp. 169–178. ACM, Bethesda (2009). <https://doi.org/10.1145/1536414.1536440>
13. Goldreich, O.: Foundations of Cryptography: volume 2, Basic Applications. Cambridge university press, Cambridge (2004). <https://doi.org/10.1017/CBO9780511721656>
14. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game. In: Proceedings of the 19th ACM Symposium on Theory of Computing, pp. 218–229. ACM, New York (1987). <https://doi.org/10.1145/28395.28420>
15. Gupta, K., Kumaraswamy, D., Chandran, N., Gupta, D.: LLAMA: a low latency math library for secure inference. In: Proceedings on Privacy Enhancing Technologies, pp. 274–294 (2022). <https://doi.org/10.56553/popets-2022-0109>
16. Jawalkar, N., Gupta, K., Basu, A., Chandran, N., Gupta, D., Sharma, R.: Orca: FSS-based secure training and inference with GPUs. In: 2024 IEEE Symposium on Security and Privacy, pp. 66–66. IEEE, San Francisco (2024). <https://doi.org/10.1109/SP54263.2024.00063>
17. Koti, N., Patra, A., Rachuri, R., Suresh, A.: Tetrad: actively secure 4PC for secure training and inference. In: Network and Distributed System Security Symposium (2022). <https://doi.org/10.14722/ndss.2022.24058>
18. Lindell, Y.: How to simulate it—a tutorial on the simulation proof technique. In: Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich, pp. 277–346 (2017). https://doi.org/10.1007/978-3-319-57048-8_6
19. Mohassel, P., Rindal, P.: ABY3: a mixed protocol framework for machine learning. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 35–52. ACM, Toronto (2018). <https://doi.org/10.1145/3243734.3243760>
20. Mohassel, P., Zhang, Y.: SecureML: a system for scalable privacy-preserving machine learning. In: 2017 IEEE Symposium on Security and Privacy, pp. 19–38. IEEE, San Jose (2017). <https://doi.org/10.1109/SP.2017.12>
21. Patra, A., Schneider, T., Suresh, A., Yalame, H.: ABY2.0: improved mixed-protocol secure two-party computation. In: 30th USENIX Security Symposium, pp. 2165–2182. USENIX Association (2021). <https://www.usenix.org/conference/usenixsecurity21/presentation/patra>
22. Ryffel, T., Tholoniati, P., Pointcheval, D., Bach, F.: AriaNN: low-interaction privacy-preserving deep learning via function secret sharing. In: Proceedings on Privacy Enhancing Technologies, pp. 291–316 (2022). <https://doi.org/10.2478/popets-2022-0015>
23. Storrier, K., Vadapalli, A., Lyons, A., Henry, R.: Grotto: screaming fast $(2+1)$ -PC for $\mathbb{Z}_2^n$ via $(2, 2)$ -DPFs. In: 2023 ACM SIGSAC Conference on Computer and Communications Security, pp. 2143–2157. ACM, Copenhagen (2023). <https://doi.org/10.1145/3576915.3623147>
24. Wagh, S., Gupta, D., Chandran, N.: SecureNN: 3-party secure computation for neural network training. In: Proceedings on Privacy Enhancing Technologies, pp. 26–49 (2019). <https://doi.org/10.2478/popets-2019-0035>
25. Wagh, S., Tople, S., Benhamouda, F., Kushilevitz, E., Mittal, P., Rabin, T.: Falcon: honest-majority maliciously secure framework for private deep learning. In: Proceedings on Privacy Enhancing Technologies, pp. 188–208 (2021). <https://doi.org/10.2478/popets-2021-0011>

26. Yang, K., Weng, C., Lan, X., Zhang, J., Wang, X.: Ferret: fast extension for correlated OT with small communication. In: 2020 ACM SIGSAC Conference on Computer and Communications Security, pp. 1607–1626. ACM, Virtual Event (2020). <https://doi.org/10.1145/3372297.3417276>
27. Yang, P., et al.: FssNN: communication-efficient secure neural network training via function secret sharing. Cryptology ePrint Archive, Paper 2023/073 (2023). <https://eprint.iacr.org/2023/073>
28. Yao, A.C.C.: How to generate and exchange secrets. In: 27th Annual Symposium on Foundations of Computer Science, pp. 162–167. IEEE (1986). <https://doi.org/10.1109/SFCS.1986.25>